

Mohamed Shafi Uzman Fassy – (102608927)

COS30031 – Task 5 – 16th August 2023

Q.1 What is the difference between a struct and a class?

The Differences between struct and a class is that in struct all members are public by default and is a commonly used data structure for grouping data together and is not very complex. Whereas, in a class, all members are private by default and classes provide encapsulation and used to define complex objects with methods that provide a controlled access to internal data.

Q.2 What are function declarations?

Function Declarations provide a way to pass information to the compiler about a function's name, the parameter types and return types before the actual implementation is encountered. They are needed for compilation order to allow functions to be called before they are defined, type checking to enable the compiler to verify parameters and return types and linkage to facilitate linking of function calls.

Q.3 Why are variable names not needed here?

The variable names are not needed here as the function declaration only focuses on the specifying functions return type and parameter types rather than specific variables names. Adding variable names can be added for clarity and documentation to aid readability of the code however these names are optional.

Q.4 Does your IDE know if this method is used?

IDEs uses features such as Highlighting the function name, Navigation options to allow jumping to the function's declaration, Analysis to locate instances, Issue Warnings to see if function is never used and Linting to detect unused functions. Therefore, these features help determine if the method is used in the code.

Q.5 un-initialised values ... what does this show and why?

So, it seems that the code is within an unreachable condition block "if (false)" so that means it would execute during runtime. showParticle(a) attempts to display an uninitialized value and the IDE might warn about it as it could be an undefined behavior.

Q.6 Did this work as expected?

showParticle(a) did not work as the debugger prompted an error that a is an uninitialized local variable unless it is fixed. Once fixed it works and displays that $x = 10$ and $y = 20$

Q.7 Initialisation list - do you know what are they?

From my understanding using an initialization list it makes it a more efficient way of initializing object members directly.

Q.8 Should show age=1, x=1, y=2. Does it?

No it does not, Instead it displays age = 1, x=2 and y=3 as we are calling getParticleWith(1,2,3).

Q.9 Something odd here. What and why?

So we are passed values to as getParticleWith(-1,2,3) where age should be -1 however in the output it's a very large number this is due to the use of unsigned int which does not hold negative values.

Q.10 showParticle(p1) doesn't show 5,6,7 ... Why?

This is because we are passing the values via parameters rather than referencing them within setParticleWith as those values would be clones of the original variables.

Q.11 So what does -> mean (in words)?

The “->” operator is used as a pointer to access struct/class members.

Q.12 Do we need to put () around *p1_ptr?

It is needed for clarity and to avoid operator precedence issues.

Q.13 What is the dereferenced pointer (from the example above)?

The dereferenced pointer could be (*p1_ptr) which gives you the object the pointer is pointing to.

Q.14 Is p1 stored on the heap or stack?

The p1 variable is stored on a stack as its been created locally within a function.

Q.15 What is p1_ptr pointing to now? (Has it changed?)

The pointer still points to the same memory address. However the data stored at the memory address has changed.

Q.16 Is the current value of p1_ptr good or bad? Explain

The current value of p1_ptr seems good so far as p1 has not gone out of scope.

Q.17 Is p1 still available? Explain.

Yes, p1 is still available as long as it's within the scope.

Q.19 Uncomment the next code line - will it compile?

It does not as it prompts that the buffer on p_array1 of size 36 bytes will be overrun and 12 bytes will be written starting at offset 36 as its out of bounds from its memory allocation.

Q.20 Does your IDE tell you of any issues? If so, how?

Yes the line is underlined and when hovering over the error mentions the above error.

Q.21 MAGIC NUMBER?! What is it? Is it bad? Explain!

The MAGIC NUMBER is a raw number in the code that and is not ideal as its not great practice.

Q.22 Explain in your own words how the array size is calculated.

The array size is calculated by taking the total size in bytes of the array in `sizeof(p_array1)` call function and dividing it by the size in bytes of a single element in the `sizeof(p_array1[0])` call function.

Q.23 What is the difference between this function signature and the function signature for `showParticleArray`?

They produce the same final output but are semantically different as the first one is a pointer to an object and the second one is a pointer to an array.

Q.24 Uncomment the following. It gives different values to those we saw before.

It gives different values to those we saw before because the function is pulling the size of the pointer which as its an array and rather than the size of the dereferenced underlying object which is the actual result.

Q.25 Change the size argument to 10 (or similar). What happens?

It caused a memory leak as the function attempts to access memory beyond its array boundary.

Q.26 What is "hex" and what does it do? (url in your notes)

Hex is an output modifier that sets the cout to be able to display hexadecimal values.

Q.27 What is new and what did it do?

There is new allocated memory for the particle on the heap as this will require manual garbage collection.

Q.28 What is delete and what did it do?

Delete deallocates memory that was allocated previously by "new" as it releases the memory on the heap to ensure that no memory leak is caused.

Q.29 What happens when we try this? Explain.

An Exception thrown error is prompted as we have come across a read access violation as we are trying to access a memory at a location that is no longer holds valid data.

Q.30 So, what is the difference between NULL and nullptr and 0?

While all 3 are similarly used to represent a null pointer, the nullptr is recommended as its used in modern C++ coding and type-safe while it avoids potential issues with 0 and NULL.

Q.31 What happens if you try this? (A zero address now, so ...)

It ended with "exited with code -1073741819" which is also known as Access Violation same as an earlier issue as I had set this if condition to true but if its set to false it exits with code 0

```
276 if (true) {  
277     showParticle((*p1_ptr));  
278 }
```

Q.32 Are default pointer values in an array safe? Explain.

As we have a memory leak it is not safe but if we set them to nullptr it is safe as we would know it points to null / nothing.

Q.33 We should always have "delete" to match each "new".

Yes, as we don't use delete to match each new the memory allocated on the heap will remain reserved and will not be released until program is terminated this leads to a memory leak.

Q.34 Should we set pointers to nullptr? Why?

Yes due to after deleting a pointer it's a good practice to set it to nullptr as this ensures that the pointer is free and points to nothing to avoid any errors if we later accidentally dereference the pointer.

Q.35 How do you create an array with new and set the size?

We Can create an array with "new" by specifying the type and within square brackets the desired size. Such as this:

```
int* Array1 = new int[sizeofarray];
```